

Epistemic Computing for Non-Associative Pharmacology: Separating Algebraic Certainty from Clinical Uncertainty

Demetrios C. Agourakis

Biomaterials and Regenerative Medicine Post-Graduate Program,
Pontifícia Universidade Católica de São Paulo (PUC-SP), Sorocaba, SP, Brazil
ORCID: 0009-0001-8671-8878

March 2026

Correspondence: demetrios@agourakis.med.br

Abstract. Recent work has established that the 7 FDA-canonical CYP450 drug-metabolising enzymes admit a Fano plane classification of their ordered triples, partitioning 343 enzyme combinations into 133 trivially order-independent, 42 associative (order-independent by algebraic structure), and 168 non-associative (predicted order-dependent) [1]. This algebraic classification is exact — it follows from the Fano plane and is not subject to measurement error. However, the *clinical* prediction derived from this classification (“will this patient’s drug cascade be order-dependent?”) depends on genotype, dosing, timing, and compliance — all subject to uncertainty. We argue that this separation between algebraic certainty and clinical uncertainty requires a programming language with *epistemic types*: types that carry uncertainty metadata through every computation, following the GUM (Guide to the Expression of Uncertainty in Measurement) standard. We describe how the Sounio programming language — a systems language with native `Knowledge<T>` types, compile-time dimensional analysis, an algebraic effect system, and refinement types — provides such a framework. We show through worked examples how the algebraic core (Fano plane classification) and the clinical layer (patient-specific predictions) coexist in the same type system, with the compiler enforcing that uncertainty is never silently discarded. This approach applies broadly to any domain where exact mathematical structure meets empirical measurement uncertainty.

Keywords: epistemic computing, uncertainty quantification, CYP450, drug interactions, type systems, GUM, non-associative algebra

MSC/ACM: 68N18, 92C45, 62P10

1. The problem: two kinds of knowledge in one computation

Consider a clinical pharmacologist analysing a three-drug cascade through CYP450 enzymes. Two fundamentally different kinds of knowledge are involved:

Algebraic knowledge (exact): “The triple CYP2D6–CYP2C9–CYP3A4 lies on a Fano line. This is an algebraic fact about $\text{PG}(2, 2)$. It is not subject to measurement error, patient variation, or experimental uncertainty.”

Clinical knowledge (uncertain): “This patient’s CYP2D6 enzyme activity is 50 ± 12 pmol/min/mg protein, based on dextromethorphan phenotyping with 95% CI. Their predicted AUC ratio under the proposed cascade is 3.2 ± 0.8 .”

In current practice, both kinds of knowledge are represented as floating-point numbers in Python, R, or MATLAB. The algebraic classification `classify_triple(6, 2, 7) = 1` and the clinical prediction `auc_ratio = 3.2` have the same type (`int` or `float`). The uncertainty ± 0.8 is tracked in comments, separate variables, or not at all.

This conflation has consequences. When algebraic results propagate into clinical predictions, the boundary between “certain because proven” and “uncertain because measured” is invisible to the program, the compiler, and often the programmer. Downstream computations silently combine exact and uncertain quantities, producing results whose confidence is unknown.

2. Background

2.1. The 168 theorem and CYP450 mapping

In companion papers [1–3], we established that:

- The number of nonzero basis associator triples in the Cayley–Dickson tower is always a multiple of $168 = |\text{PSL}(2, 7)|$.
- The 7 FDA-canonical CYP450 isoforms map to the Fano plane, classifying 343 enzyme triples into 133 trivial, 42 associative, and 168 non-associative.
- The prediction is testable: DDI order-dependence should correlate with the Fano classification.

The algebraic layer is exact and verified exhaustively through dimension 64. The clinical layer is where uncertainty enters.

2.2. The GUM framework

The *Guide to the Expression of Uncertainty in Measurement* (GUM) [4] provides the international standard for propagating measurement uncertainty through calculations. For a function $f(x_1, \dots, x_n)$ of measured quantities x_i with standard uncertainties $u(x_i)$, the combined standard uncertainty is:

$$u(f) = \sqrt{\sum_{i=1}^n \left(\frac{\partial f}{\partial x_i} \right)^2 u(x_i)^2} \quad (1)$$

GUM uncertainty propagation is mathematically well-defined but rarely enforced by programming tools. Most implementations require manual tracking of uncertainty through separate variables.

3. Epistemic types in Sounio

The Sounio programming language [5] introduces `Knowledge<T>` as a first-class type. A value of type `Knowledge<T>` carries:

- A *central value* of type `T`
- A *standard uncertainty* u (following GUM)
- An optional *provenance* tag identifying the uncertainty source

Arithmetic on `Knowledge<T>` values automatically propagates uncertainty via Equation 1. The compiler enforces that:

1. `Knowledge<T>` values cannot be silently cast to `T` (uncertainty cannot be discarded).
2. Exact values (type `T`) can be promoted to `Knowledge<T>` with $u = 0$.
3. Dimensional analysis is enforced at compile time via the unit system.

3.1. Example: algebraic certainty

```
// The Fano plane classification is exact.
// classify_triple returns i32, not Knowledge<i32>.
// There is no uncertainty to track.
```

```
let classification: i32 = classify_triple(cyp_2d6(), cyp_2c9(), cyp_3a4())
// = 1 (ASSOCIATIVE). Compile-time type: i32. No  $\pm$ . Algebraic fact.
```

The return type `i32` carries zero uncertainty by construction. The compiler knows this value is exact.

3.2. Example: clinical uncertainty

```
// CYP2D6 activity measured by dextromethorphan phenotyping.
// The measurement has uncertainty from assay precision.

let cyp2d6_activity: Knowledge<pmol_per_min_per_mg>
  = measure(50.0, uncertainty: 12.0)

// Drug clearance depends on enzyme activity.
// Uncertainty propagates automatically (GUM).

let clearance: Knowledge<mL_per_min>
  = intrinsic_clearance(cyp2d6_activity, substrate_km)
// clearance.value = 23.4, clearance.uncertainty = 5.6

// AUC ratio for the drug cascade.
let auc_ratio: Knowledge<f64> = compute_auc_ratio(clearance, dose, bioavailability)
// auc_ratio.value = 3.2, auc_ratio.uncertainty = 0.8
```

Every intermediate value carries its uncertainty. The compiler rejects any attempt to use `auc_ratio` as a plain `f64` — the uncertainty must be explicitly acknowledged (e.g., by checking a confidence threshold or propagating it further).

3.3. Example: the bridge — algebra meets clinic

// THE KEY COMPUTATION: combine algebraic certainty with clinical uncertainty.

```
let triple_class: i32 = classify_triple(cyp_a, cyp_b, cyp_c)
// Exact. No uncertainty.

let patient_response: Knowledge<f64> = if triple_class == 2 {
  // NON-ASSOCIATIVE triple: predict order-dependence
  // Clinical prediction with uncertainty from genotype + dosing
  predict_order_dependence(patient_genotype, cascade_timing)
} else {
  // ASSOCIATIVE triple: predict order-independence
  // Still has uncertainty (from patient factors), but lower
  predict_order_independence(patient_genotype)
}

// The compiler knows:
//   triple_class has type i32 (exact, algebraic)
//   patient_response has type Knowledge<f64> (uncertain, clinical)
// They coexist. Neither contaminates the other.
```

The algebraic classification gates the clinical computation, but the types remain distinct. The `if` branch is determined by exact algebra; the result carries clinical uncertainty. A downstream function receiving `patient_response` knows, from its type alone, that this value is uncertain and must be treated accordingly.

4. Additional type-system features

4.1. Compile-time units

Sounio's unit system prevents dimensional errors at compile time:

```
let dose: mg = 500.0
let volume: L = 0.35
let concentration: mg_per_L = dose / volume // type-checked
// let error = dose + volume                // COMPILE ERROR: mg + L
```

For CYP450 pharmacokinetics, this means clearance rates (mL/min), AUC values (ng·h/mL), and doses (mg) cannot be accidentally combined without dimensional consistency.

4.2. Effects

Sounio's algebraic effect system tracks computational side effects:

```
fn classify_triple(a: i32, b: i32, c: i32) -> i32 with Mut, Panic {
  // Pure computation. No IO, no network, no randomness.
  // The 'with' clause proves this function's result depends
  // only on its inputs – it is algebraically deterministic.
}

fn predict_order_dependence(g: Genotype, t: Timing)
  -> Knowledge<f64> with Mut, Panic, IO {
  // This function may read patient data (IO effect).
  // The compiler enforces that callers handle the IO effect.
}
```

The effect system makes the boundary between pure mathematics and empirical data visible at the type level.

4.3. Refinement types

```
type BasisIndex = { x: i32 | x >= 1 && x <= 7 }
type FanoLine = { t: [BasisIndex; 3] | is_on_fano_line(t[0], t[1], t[2]) }
```

The compiler rejects any attempt to construct a FanoLine from elements that don't form a valid Fano line. The algebraic structure is enforced at compile time, not by runtime checks.

4.4. Linear types for drug doses

```
linear struct Dose { amount: Knowledge<mg>, drug: DrugId }

fn administer(patient: &!Patient, dose: Dose) with IO {
  // 'dose' is consumed exactly once.
  // The linear type ensures:
  //   - The dose cannot be duplicated (double-dosing prevented)
  //   - The dose cannot be silently dropped (missed dose detected)
}
```

A drug dose is a *linear resource*: it must be administered exactly once. The compiler enforces this at the type level, preventing a class of medication errors that no runtime check can catch.

5. The epistemic pipeline: from algebra to clinic

Putting these features together, the full pipeline from mathematical theorem to clinical prediction is:

Layer	Type	Uncertainty
Fano plane classification	i32	None (exact)
Enzyme activity measurement	Knowledge<pmol/min/mg>	GUM from assay
Genotype probability	Knowledge<f64>	Population frequency
Drug clearance	Knowledge<mL/min>	Propagated from enzyme + genotype
AUC ratio prediction	Knowledge<f64>	Propagated from clearance + dose
Order-dependence prediction	Knowledge<f64>	Propagated from all above

Table 1: The epistemic pipeline. Each layer has a type that encodes its uncertainty status. The compiler enforces that no layer silently discards uncertainty from upstream.

The first layer (Fano classification) is exact. Every subsequent layer introduces or propagates uncertainty. The final clinical prediction carries the combined uncertainty from every measurement and model

parameter, following GUM. At no point can the programmer accidentally treat an uncertain quantity as exact — the type system prevents it.

6. Discussion

The approach described here is not specific to CYP450 pharmacology. Any domain where exact mathematical structure meets empirical measurement uncertainty would benefit from epistemic types. Examples include:

- **Crystallography**: space group classification (exact) combined with diffraction intensity measurement (uncertain).
- **Genomics**: sequence alignment scoring (algorithmically exact) combined with base-calling quality (uncertain).
- **Climate modelling**: conservation laws (exact) combined with parameterised sub-grid processes (uncertain).

The key insight is that uncertainty is not a property of numbers — it is a property of *how we know* those numbers. A programming language that tracks *how we know* alongside *what we know* prevents a class of errors that no amount of runtime validation can catch: the silent conflation of certainty with ignorance.

The 168 theorem provides an unusually clean example: the algebraic classification of drug interaction triples is provably exact (it follows from the Fano plane, verified exhaustively [3]), while the clinical prediction derived from that classification depends on patient-specific measurements with irreducible uncertainty. Sounio’s type system holds both, without confusing them.

7. Availability

The Sounio programming language, including the `Knowledge<T>` type, the CYP450 module (`stdlib/medical/cyp450_fano.sio`), and the genetic code module (`stdlib/medical/genetic_code_168.sio`), is available at github.com/agourakis82/sounio.

Bibliography

1. Agourakis, D. C.: PG(k−1, 2) Geometry as a Mathematical Framework for Order-Dependent Processes in Biology, (2026)
2. Agourakis, D. C., Gerenutti, M.: The 168 Count for Octonion Basis Associators and a Computational Cayley–Dickson Tower Extension. *Advances in Applied Clifford Algebras*. (2026)
3. Agourakis, D. C.: The 168 Quantum of Non-Associativity in the Cayley–Dickson Tower, (2026)
4. JCGM: Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement. Joint Committee for Guides in Metrology (2008)
5. Agourakis, D. C.: Sounio: A systems programming language for epistemic computing, <https://github.com/agourakis82/sounio>